COMPUTER ARCHITECTURE · PROJECT WORK

CPU/GPU Bottleneck Analysis: A Parallel Approach for Searching in a Binary Tree

From POSIX threads on a quad-core CPU to CUDA on an NVIDIA Tesla T4

Alessio Simoncini · Gabriele Caioli

Master's Degree in Computer Engineering · University of Pisa

The problem

Data structure

A **binary search tree**: every parent is greater than its left child and smaller than its right child.

Duplicate values are not allowed.

Algorithm

1. Insert the values of a vector into the tree.
2. Search every value of a second, much larger vector.
3. Return a vector of booleans: found or not found.

INTRODUCTION

Workloads and goals

Workloads: values searched

Workload	Values
1 · Low	10,000
2 · Medium	100,000
3 · High	1,000,000
4 · Very high	10,000,000

PART 1 · CPU GOAL

11 M values/s

with predictable scaling as the number of threads grows

PART 2 · GPU GOAL

1 G values/s

one billion values searched per second

PART 1

CPU · POSIX threads

Three versions on a quad-core Intel Core i5, profiled with Intel VTune

Processor

4 cores

8 threads

Quad-core Intel Core i5, 10th generation, in a laptop

Intel Core i5-10210U	
Cores / threads	4 / 8
Base / turbo clock	1.6 GHz / 4.2 GHz
Typical TDP	15 W
L1 cache	4 × 32 KB instr. + 4 × 32 KB data
L2 cache	4 × 256 KB
L3 cache	6 MB

A range of values for each thread

Each thread owns the subtree rooted at one node of level $\log_2 N$, and therefore a range of values. Each value to search is assigned to the thread whose range contains it.

1

Insert

Build the tree from the vector of values.

2

Dispatch

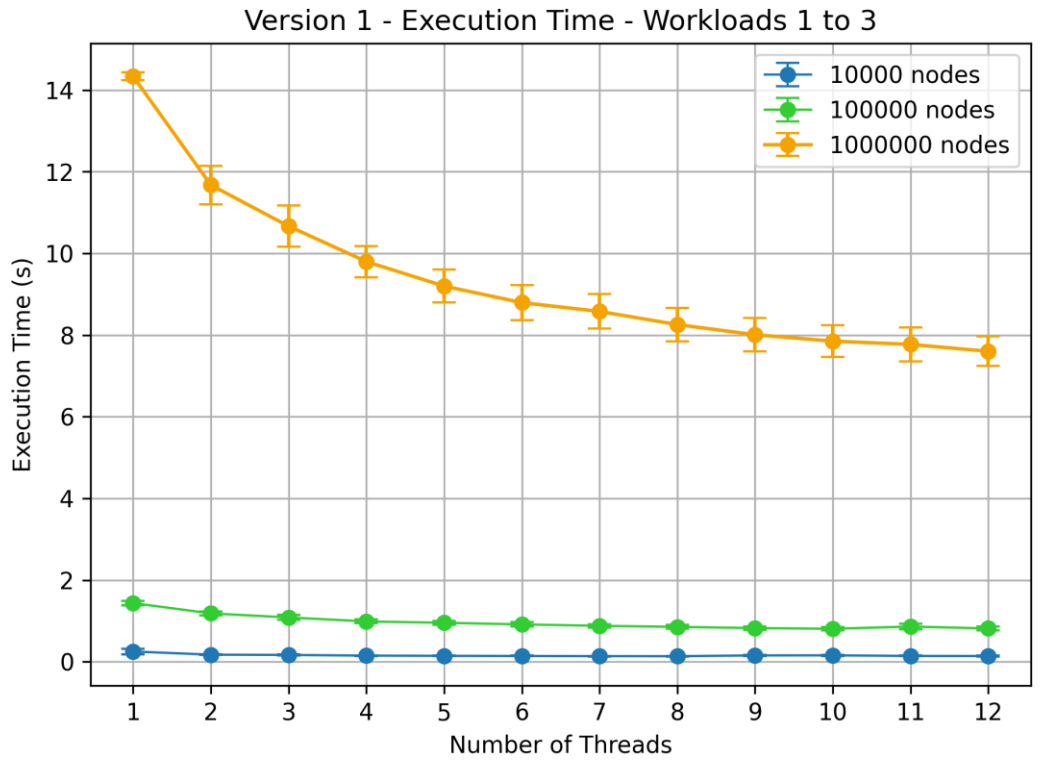
The main thread walks the top levels of the tree and hands each value to the thread that owns its range.

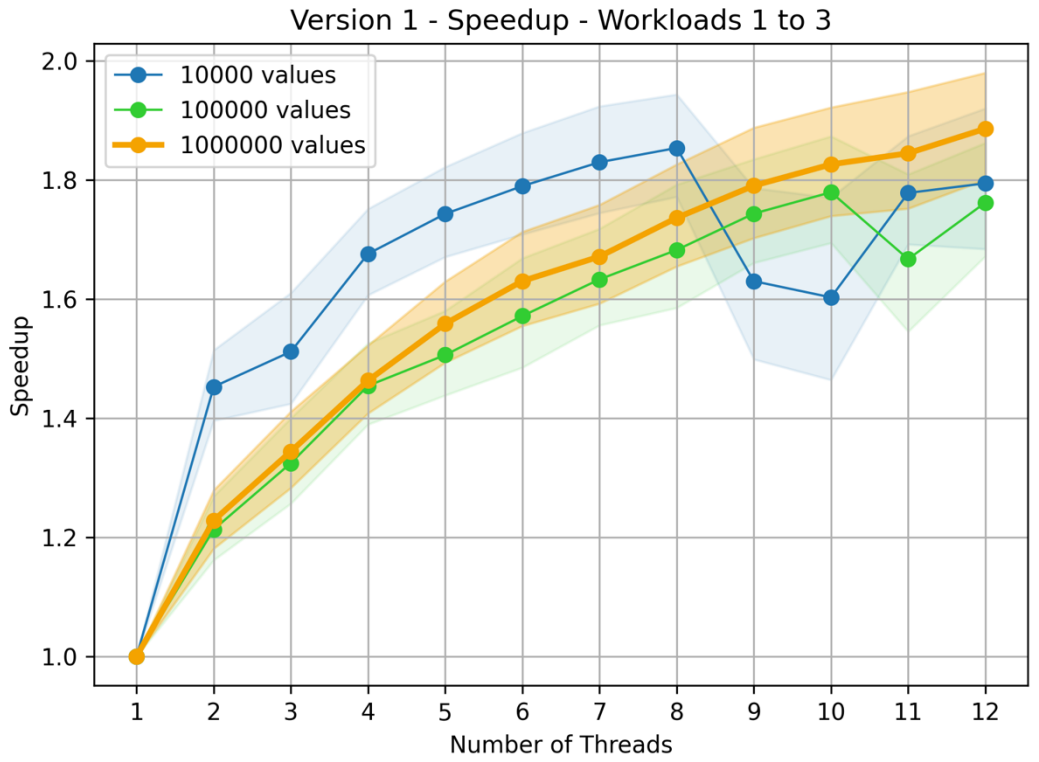
3

Search

The worker searches its own subtree and writes the result.

Performance





8.1 s

at 8 threads, 1M values

125,000 values/s

throughput at 8 threads

Weak parallelization: a small gain for each thread up to 8, and odd behaviour on the larger workloads.

Bottleneck: waiting on a semaphore

Sync object	Wait time	Wait count
Semaphore 0x05d9ea5e	**14.628 s**	**99,655**
Semaphore 0x98b3cc5f	1.018 s	57,017
Multiple objects	0.160 s	33
Critical section 0x4638ad52	0.088 s	53
Critical section 0xd9128f53	0.055 s	13

The main thread blocks whenever it assigns a search to a thread that has not finished the previous one.

This happens every time **two or more consecutive values** fall in the **same range**.

Split the values before searching

Before the searches start, the vector of values is divided into sub-vectors, one per thread, according to the range of values each thread covers.

Independent threads

Each thread gets its own sub-vector and never waits for the others.

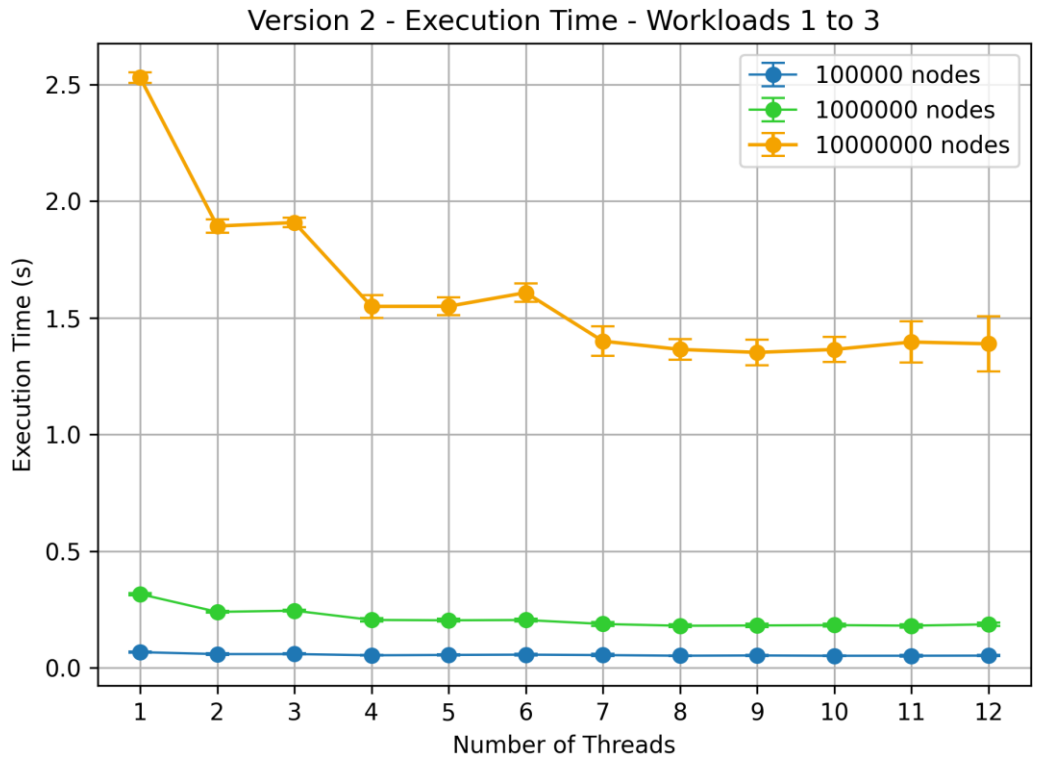
Own subtree

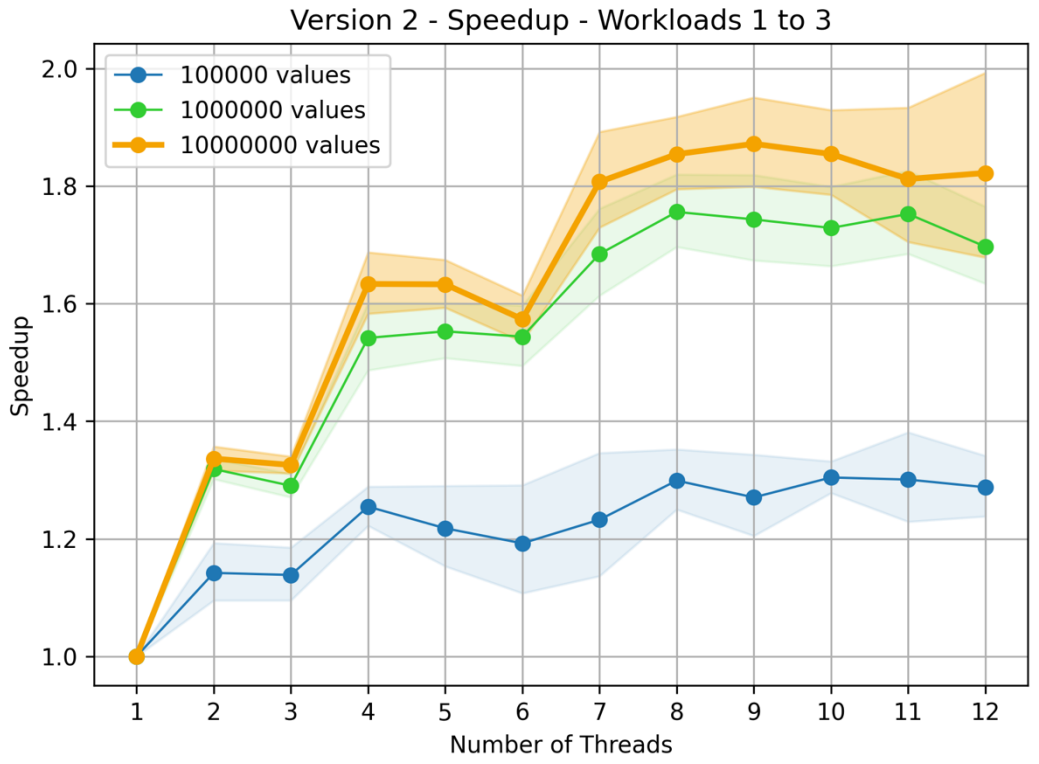
Each thread searches only in the subtree that holds its range.

Extra overhead

Splitting the vector costs time, and it is included in the measurements.

Performance





1.4 s

at 8 threads, 10M values

40× faster

1M values: 0.2 s, was 8.1 s

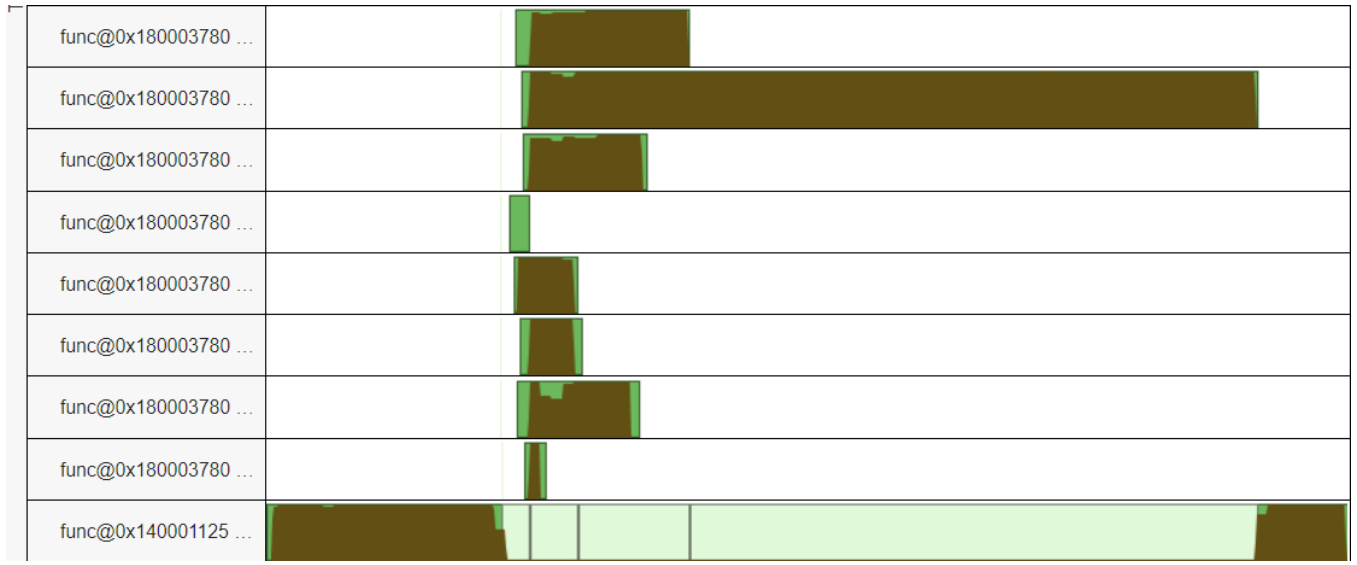
7.4 M values/s

60× the throughput of V1

Larger workloads now scale better, but the speedup stays low and drops after each power of 2.

Bottleneck: an unbalanced tree

Usage of the 8 logical cores over time



The tree is **not balanced**: the load of each thread depends on how many values fall in its subtree, so a few threads work while the others are idle.

Equal shares, every search from the root

Before the searches start, the vector of values is logically divided into sub-vectors of equal size, one per thread.

Same load

Every thread has the same number of values to search.

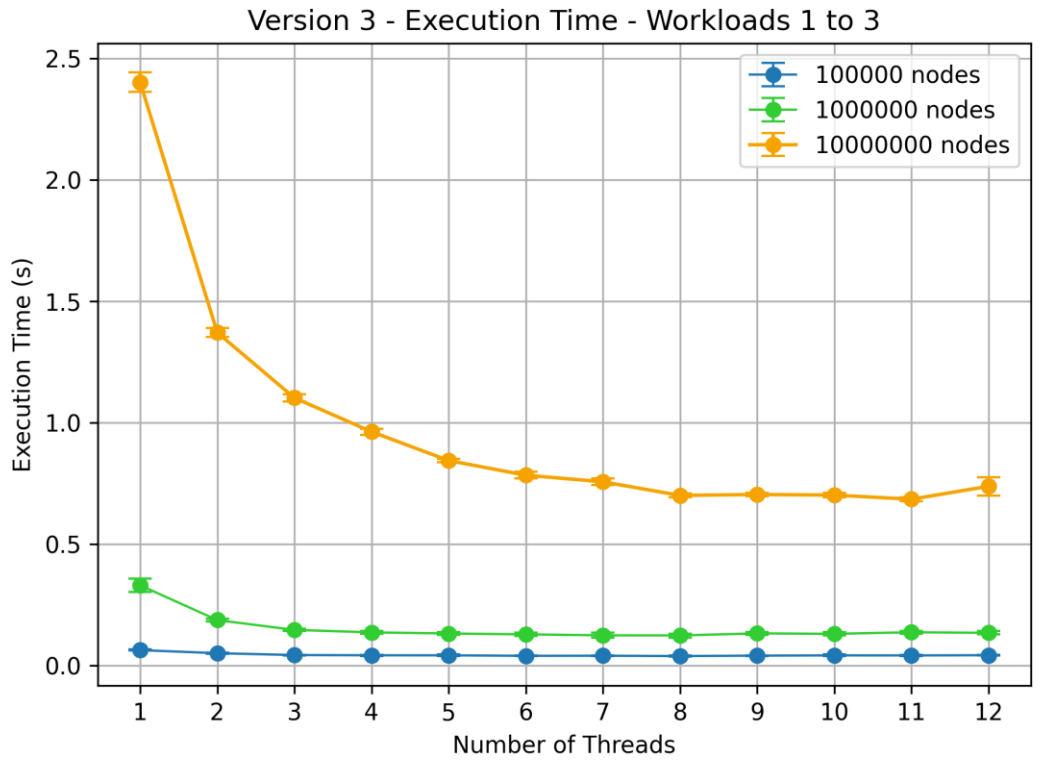
Whole tree

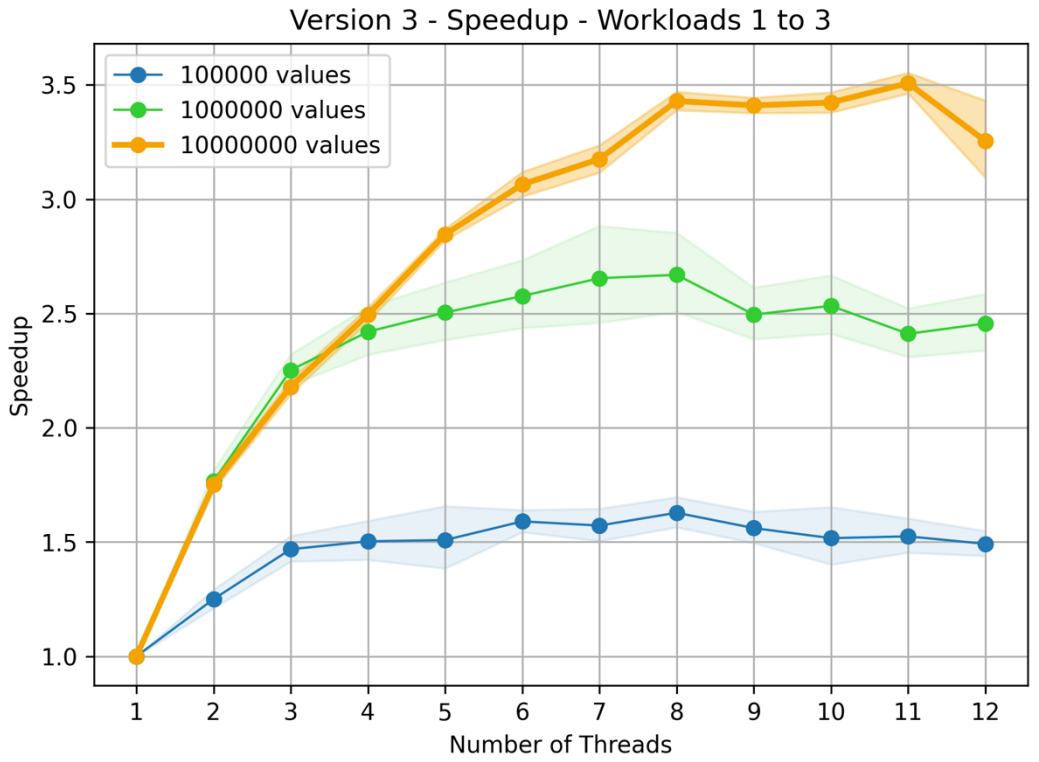
Every thread starts its searches from the root of the tree.

Less overhead

No ranges or subtrees to compute and manage anymore.

Performance





0.7 s

at 8 threads, 10M values

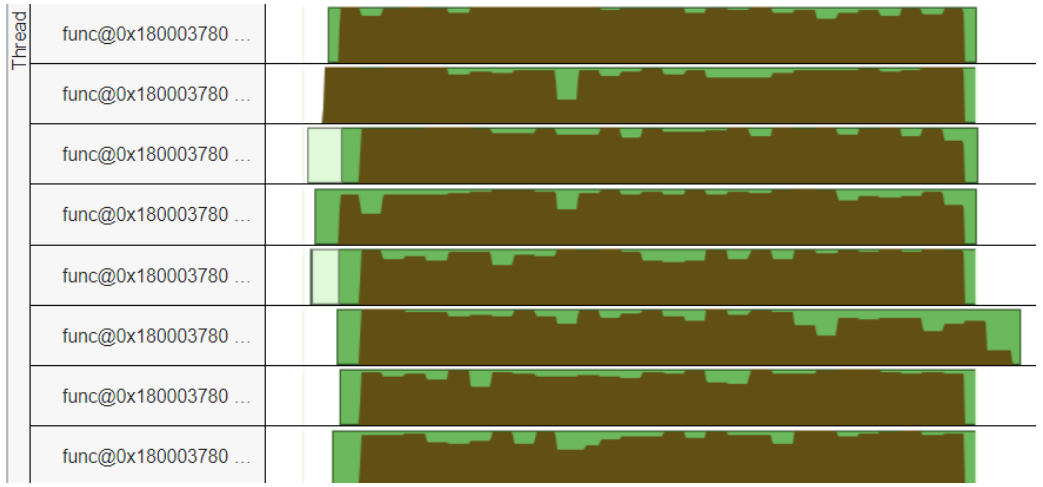
14.2 M values/s

about 2× the throughput of V2

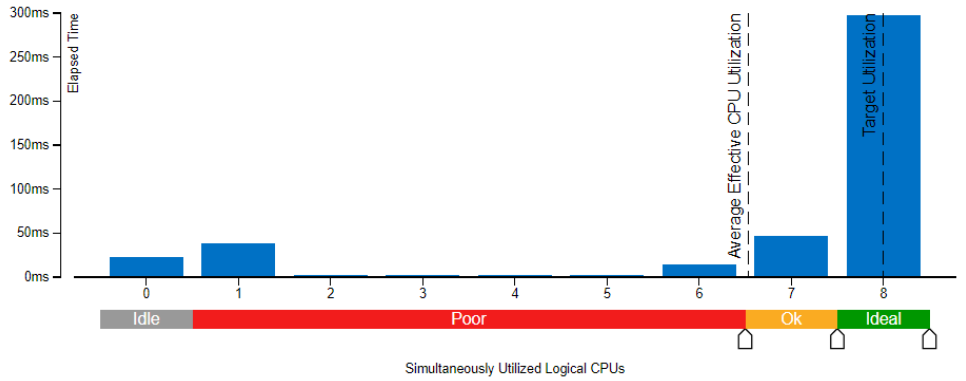
The maximum speedup finally reaches a reasonable value.

All eight logical cores at work

Usage of each logical core over time

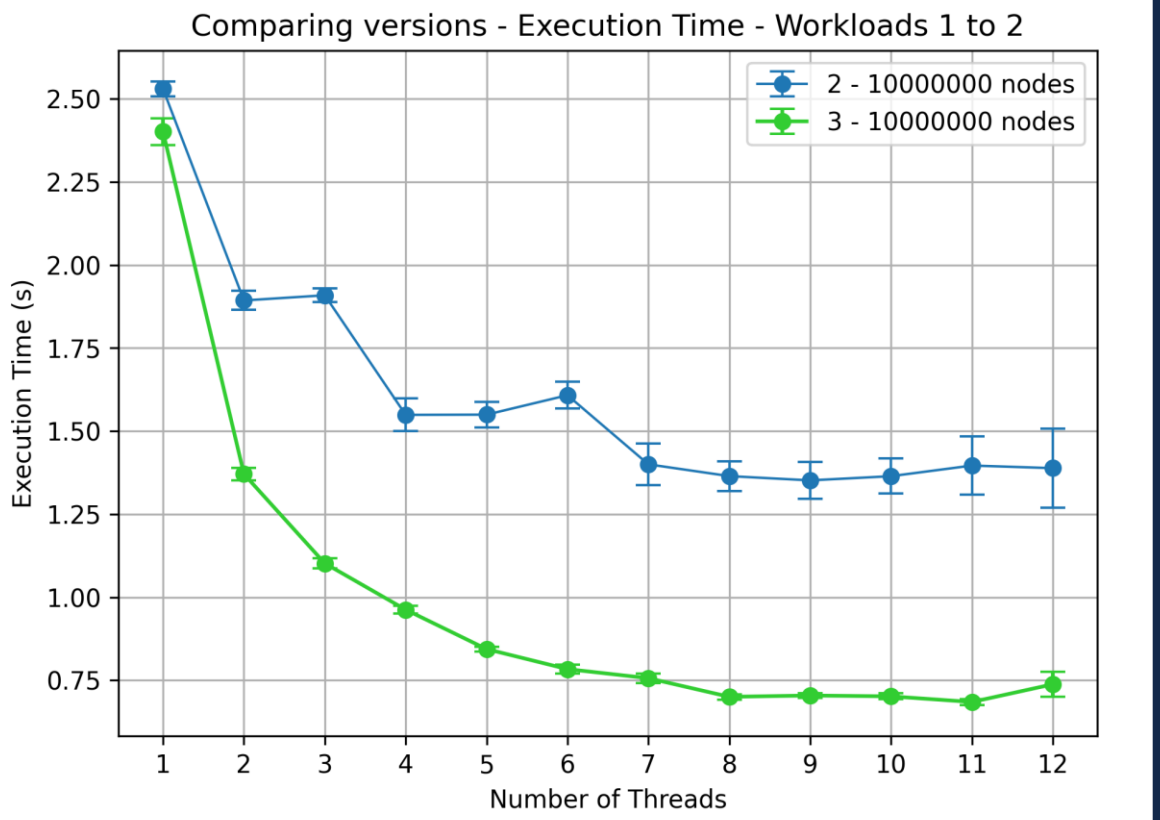


Time by number of cores used at once



With equal shares the load is balanced: most of the time all the logical cores are busy at the same time.

Throughput at 8 threads



Version 1	**125,000 /s**
Version 2	**7.4 M /s**
Version 3	**14.2 M /s**

Goal: 11 M values/s — **reached** by Version 3, about 114× the throughput of Version 1.

PART 2

GPU · CUDA

Four versions on an NVIDIA Tesla T4, profiled with Nsight Compute

NVIDIA Tesla T4

Memory

Property	Value
Size	16 GB
Type	GDDR6
Bus	256 bit
Bandwidth	320 GB/s

Compute

Property	Value
Streaming multiprocessors	**40**
Shading units	2,560
Tensor cores	320
L1 cache	64 KB per SM
L2 cache	4 MB

How many threads run at once?

Device attribute	Value
Max warps per multiprocessor	32
Max warps per scheduler	8
Warp size	32
Max threads per block	1,024
Max threads per multiprocessor	1,024

40 SMs × 32 warps × 32 threads

40,960 threads

2^{15} (32K) < 40,960 < 2^{16} (64K): we expect the best performance at **2^{15} threads**.

The final CPU algorithm, on the GPU

The tree built on the CPU is copied to the GPU node by node, keeping its pointers. The search works like CPU Version 3.

Logical split

The vector of values is divided among the threads, as on the CPU.

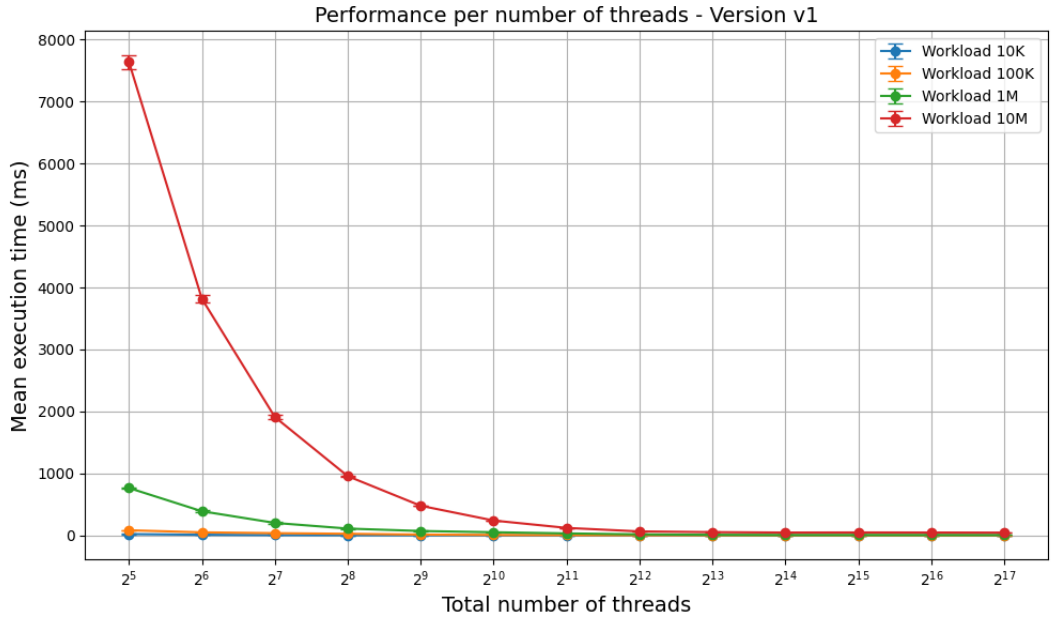
From the root

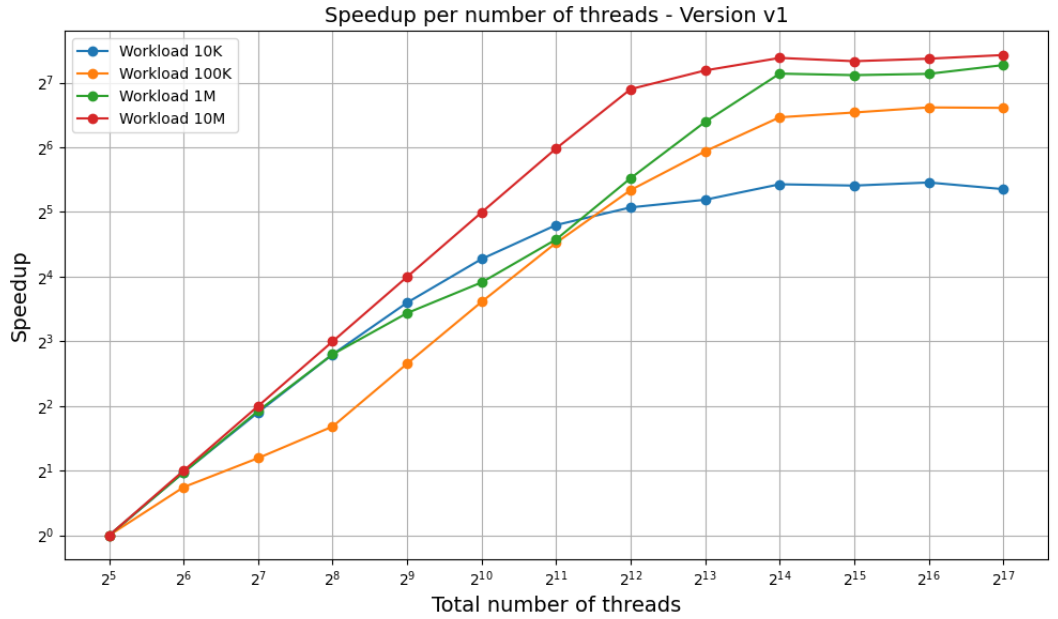
Every thread starts each search from the root of the tree.

32 threads per block

Fixed block size: one warp per block.

Performance





47.4 ms

at 2^{15} threads, 10M values

211.0 M values/s

throughput at 2^{15} threads

Already about 15× the best CPU version, but still far from the goal.

Bottlenecks: memory throughput and cache misses

Low memory throughput

Metric (% of peak)	Value
Compute memory access	0.21%
Compute memory request	0.21%
DRAM	0.09%
L1/TEX cache	0.42%
L2 cache	0.08%
SM memory	0.94%

Many L1 cache misses

Global load sectors	Count	Share
Total	3,277	100%
Hits	2,337	71%
Misses	**940**	**29%**

The cause: a tree made of pointers

STRUCTURE

Binary tree with a pointer structure

LAYOUT

Nodes scattered in memory: non-sequential, unpredictable accesses

EFFECT

Non-coalesced accesses and cache misses

From a tree of pointers to an array of structs

```
// Version 1: tree node
typedef struct Node {
    uint32_t value;
    struct Node* left;
    struct Node* right;
} Node;
```



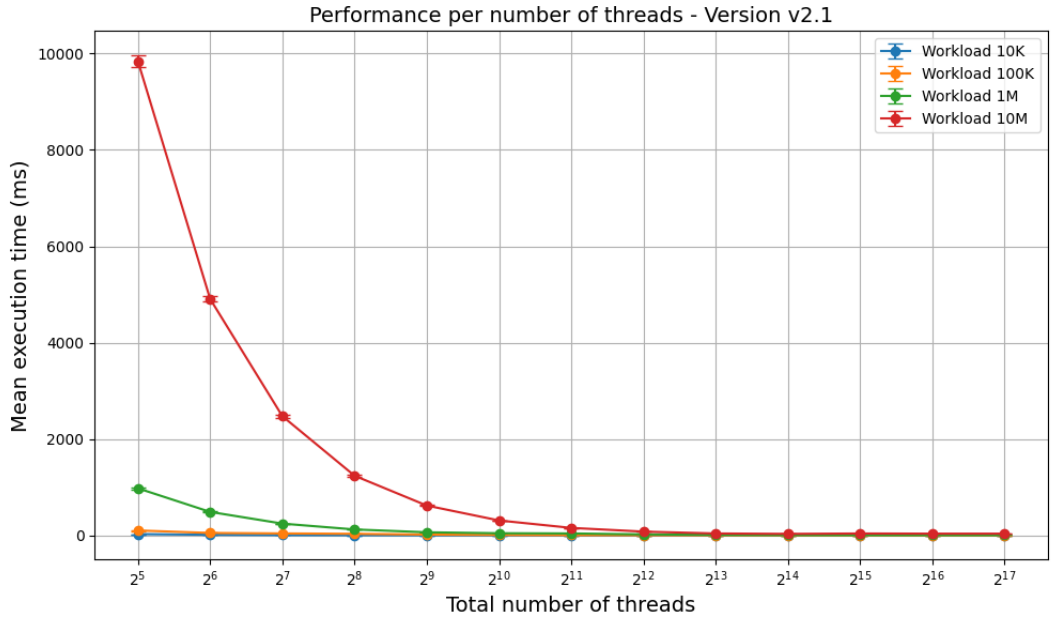
```
// Version 2.1: array element
typedef struct GpuNode {
    uint32_t value;
    int left_child_idx;
    int right_child_idx;
} GpuNode;
```

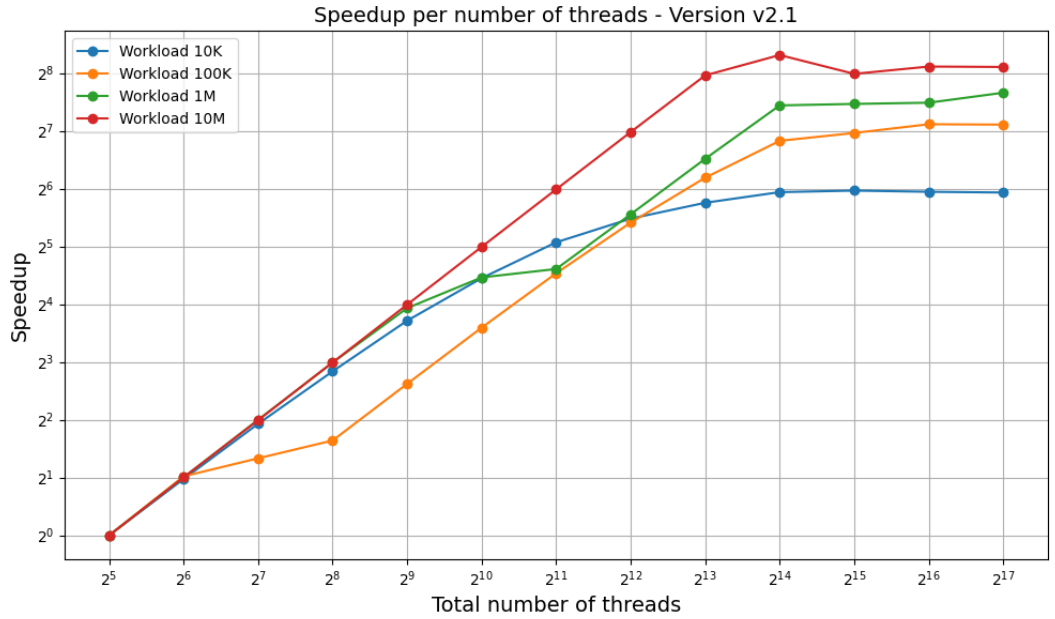
24 bytes

12 bytes

The whole tree becomes one contiguous array: **less memory** and **no pointers**, children are indices into the array.

Performance





38.9 ms

at 2^{15} threads, 10M values

257.3 M values/s

throughput at 2^{15} threads

+18%

over Version 1

Better cache use, higher memory throughput

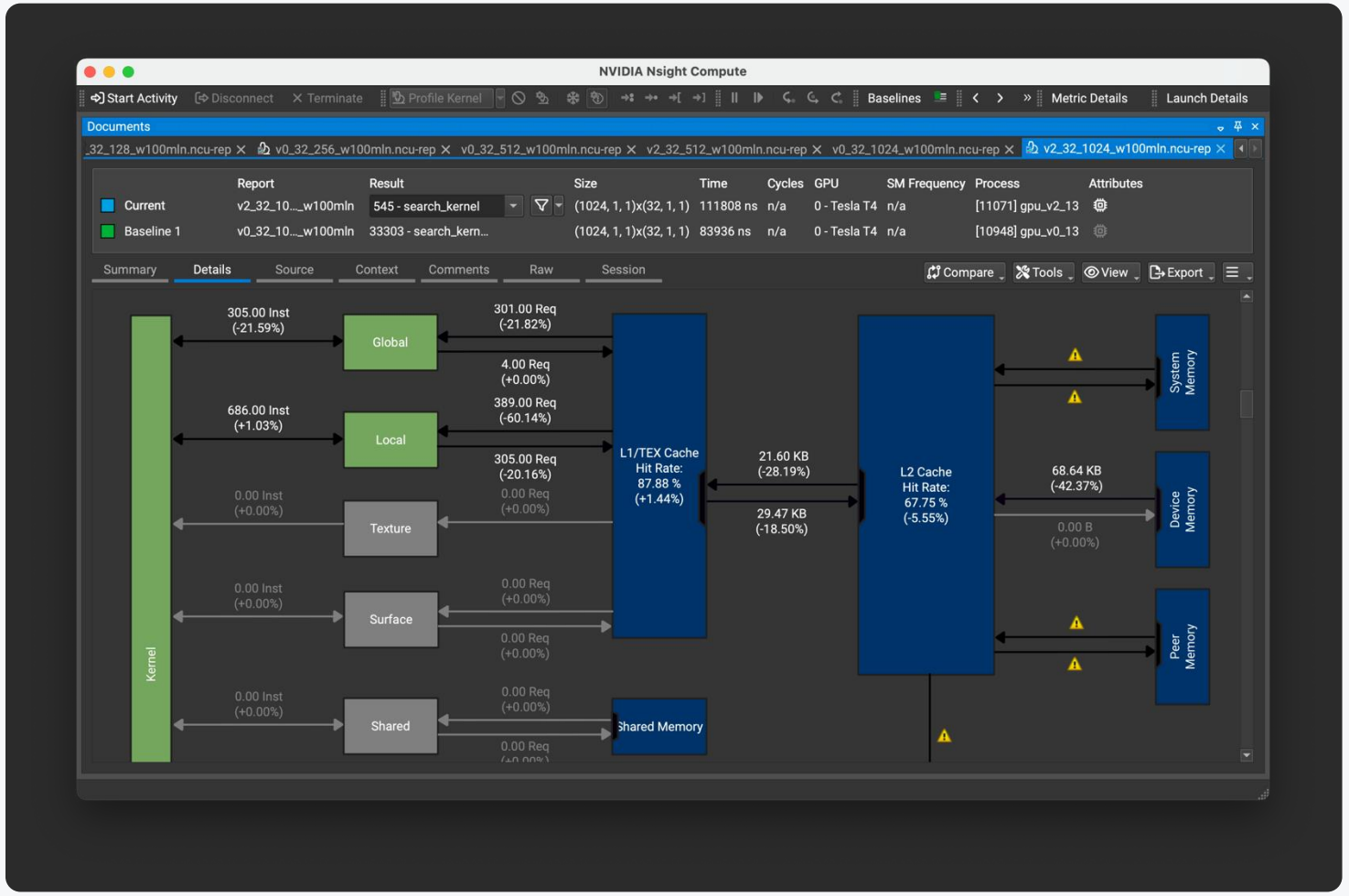
L1 hit rate on global memory: 71% → 80%

Load sectors	V1	V2.1
Total	3,301	3,163 (−4.18%)
Hits	2,323	2,532 (+9.00%)
Misses	**978**	**631 (−35.48%)**

Memory throughput (% of peak)

Metric	V2.1 (vs V1)
Compute memory access	0.29% (+39.72%)
Compute memory request	0.28% (+32.04%)
DRAM	0.14% (+68.11%)
L1/TEX cache	0.59% (+39.72%)
L2 cache	0.12% (+53.35%)
SM memory	1.11% (+17.60%)

Fewer memory requests



Memory load is reduced along the whole hierarchy.

More throughput + fewer requests = more coalesced accesses

An array that is the tree

The tree is rebalanced, then each array element holds the value of one node, in **breadth-first order**. The structure lives in the indices:

Node of index i	Index
Left child	`2 * i + 1`
Right child	`2 * i + 2`
Parent	`(i - 1) / 2`

SIZE OF A NODE

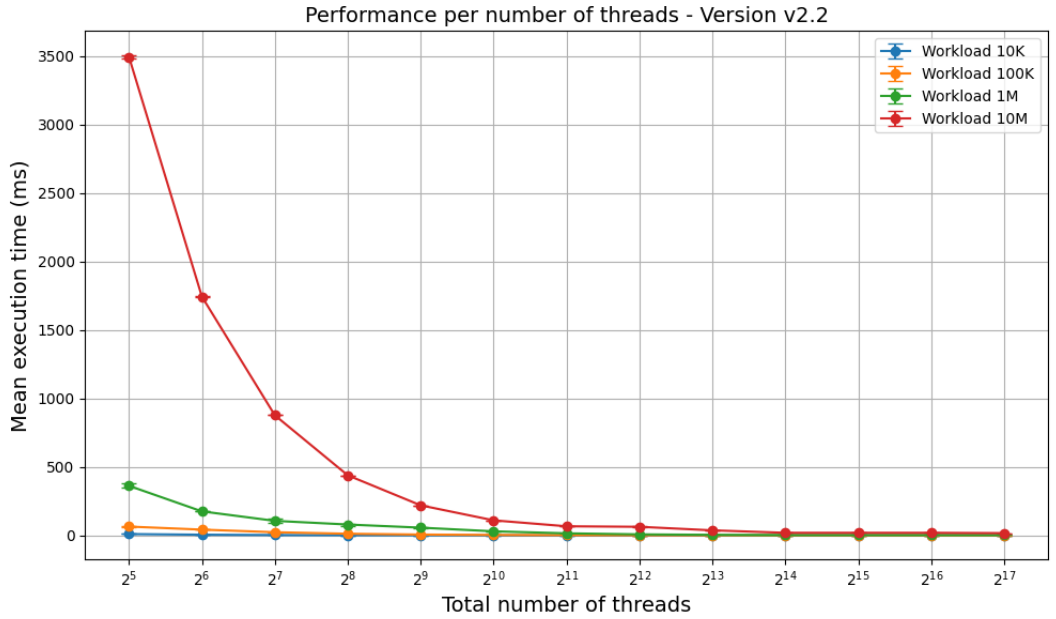
24 B → 4 B

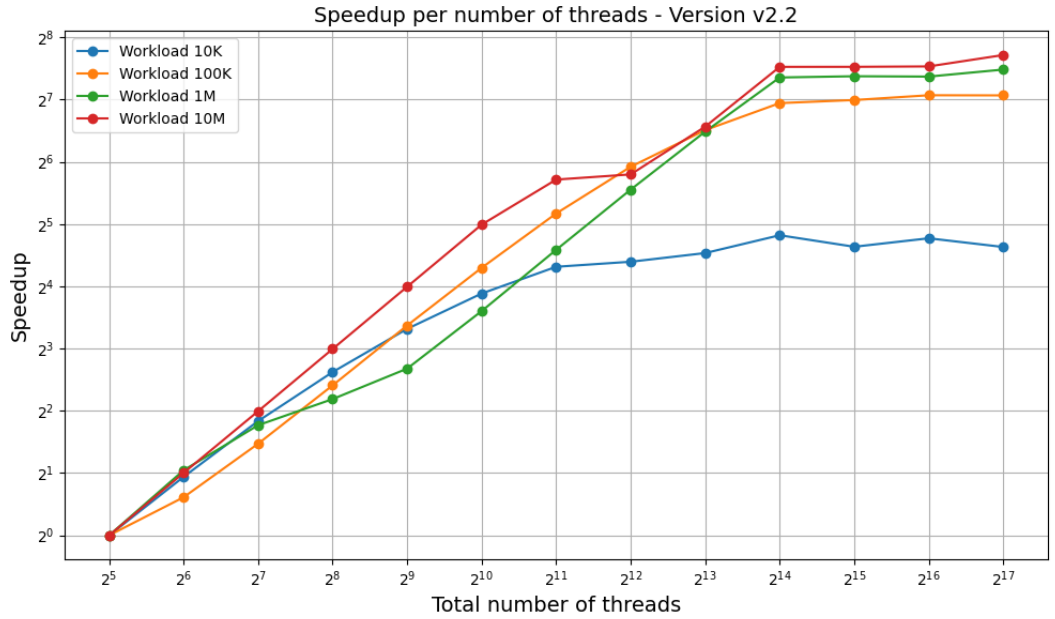
about 83% less memory



Root, then level 1, then level 2…: no pointers and no indices to store.

Performance





19 ms

at 2^{15} threads, 10M values

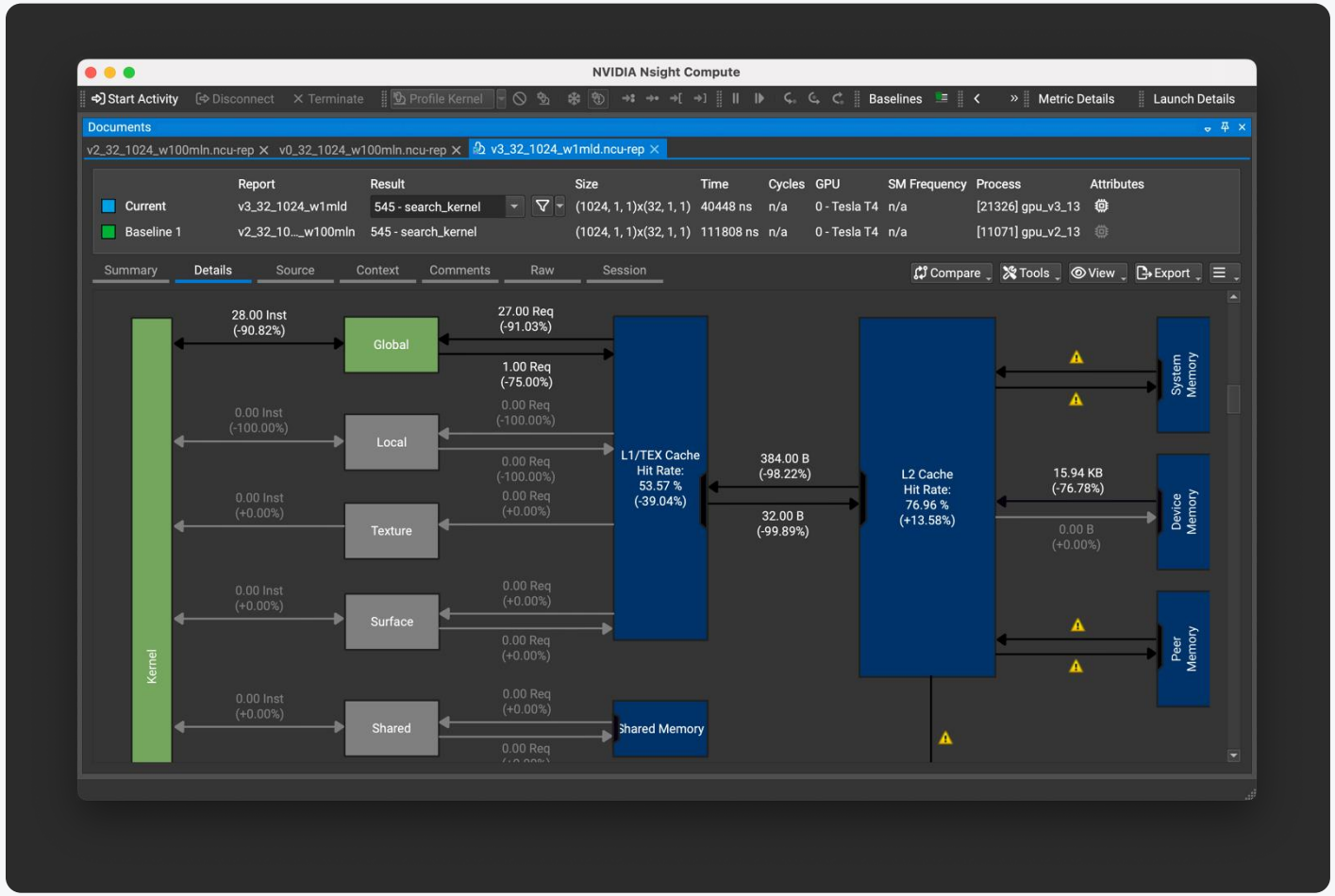
526.3 M values/s

throughput at 2^{15} threads

more than 2×

Version 2.1

Far fewer requests to global memory

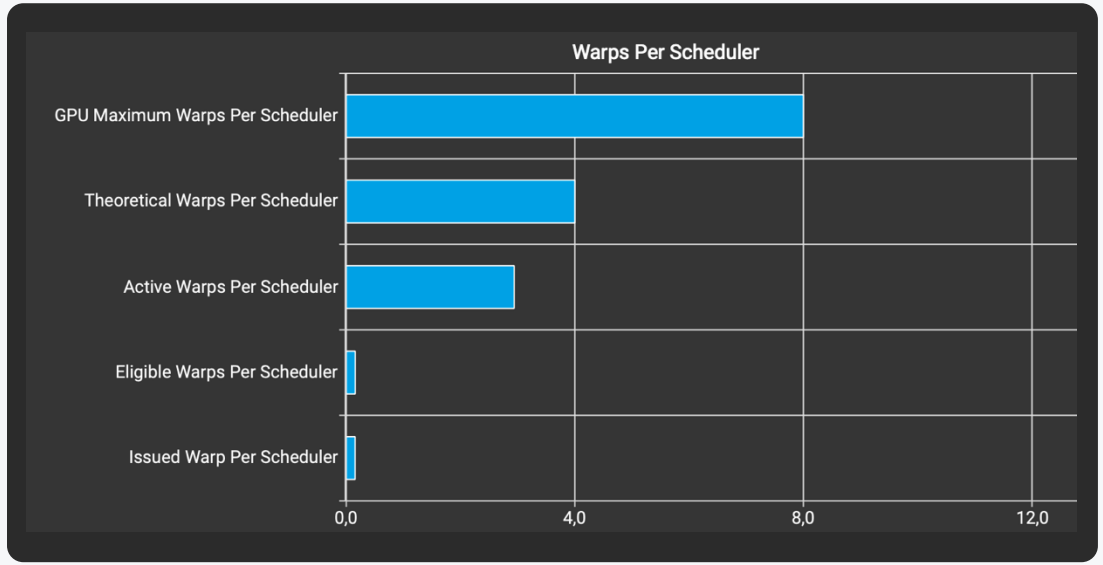


DRAM metric	Value
Active cycles (avg)	331 (−16.31%)
Elapsed cycles (avg)	213,504 (−53.67%)
Sectors read	662 (−16.31%)

A smaller number of requests, each one larger in scope.

Bottleneck: every step waits for global memory

```
// Found
if (gpu_tree[current] == target)
    return 1;
// Otherwise move to the correct child
current = (target < gpu_tree[current])
    ? (current * 2 + 1)
    : (current * 2 + 2);
```



Both reads: address space **Global**, 32-bit loads, 50% excessive L2 sectors.

The next index depends on a value of the tree, which is in **slow global memory**.

And not all the theoretical warps are used.

The whole tree in shared memory

At the start of the kernel every block copies the tree array into its **shared memory**: the walk down the tree never touches global memory again.

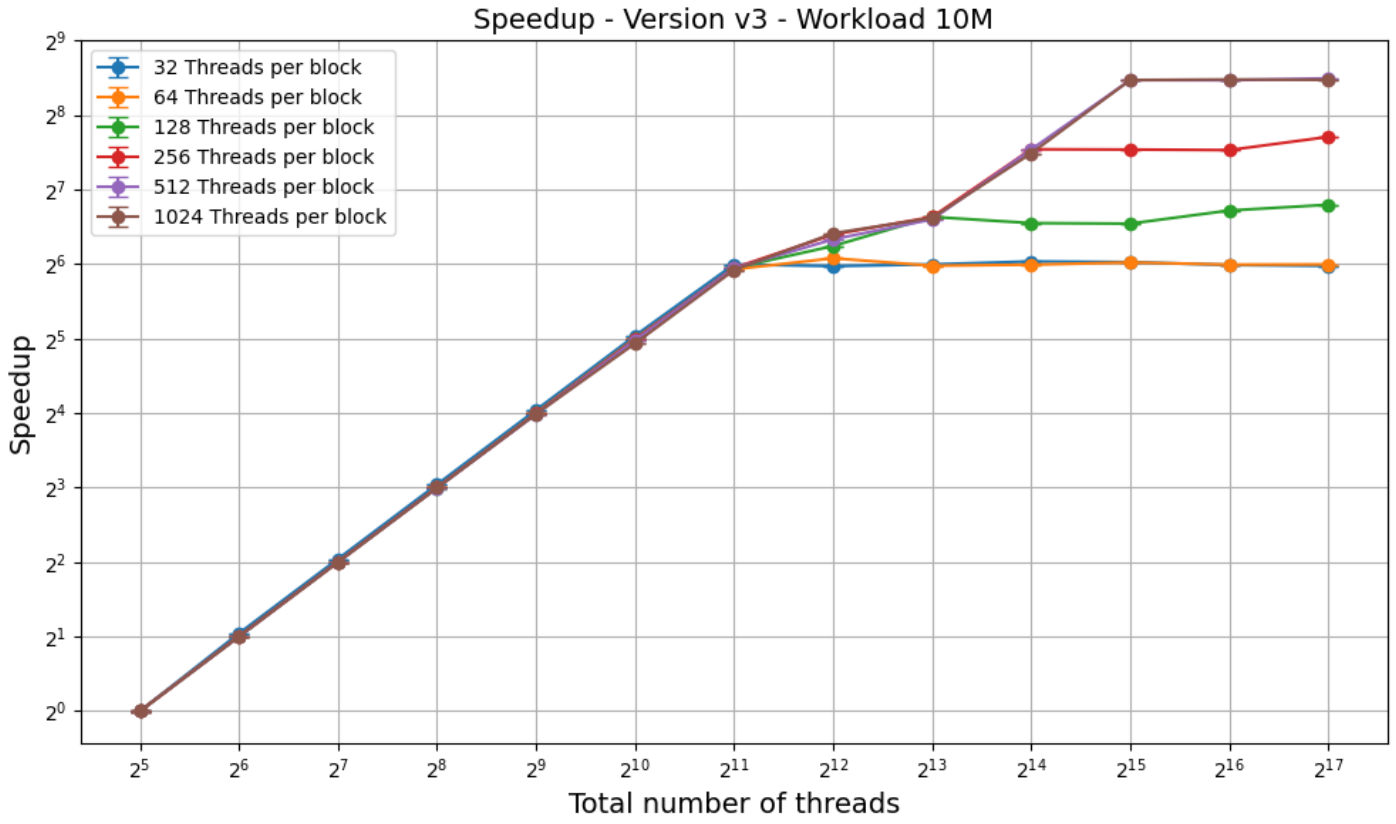
Max shared memory	Bytes
Per block	49,152
Per block (opt-in)	65,536
Per multiprocessor	65,536

SIZE OF THE TREE

32 KB

8,190 nodes ($\approx 2^{13}$) × 4 B = 2^{15} B: it fits in the shared memory of a block.

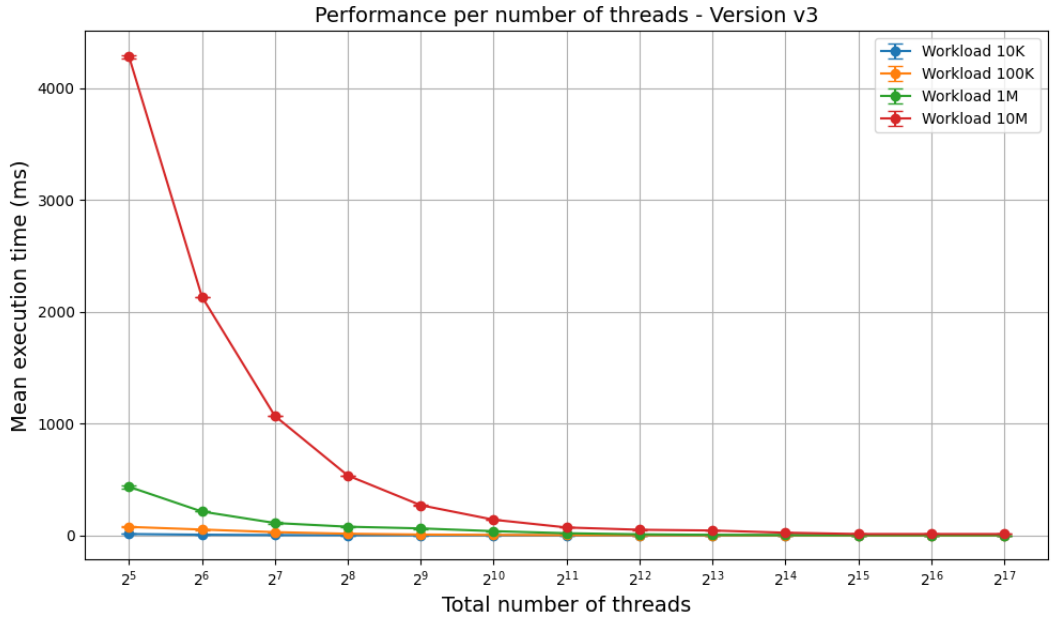
Choosing the block size

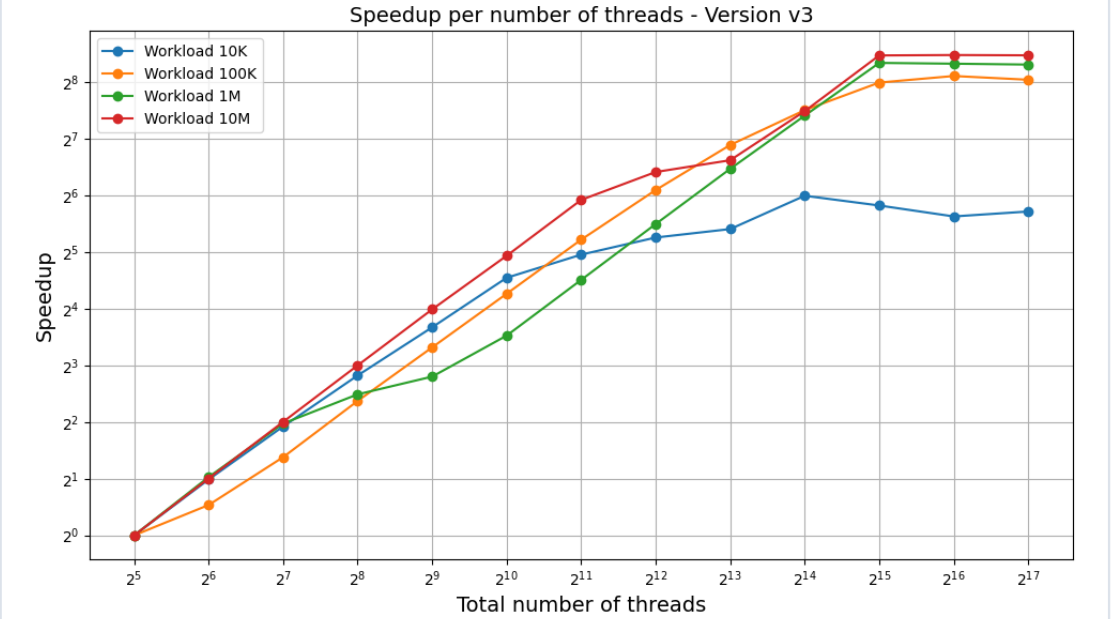


Same total number of threads, different block sizes and numbers of blocks, on 10M values.

Small blocks stop scaling early; with **1,024 threads per block** the speedup keeps growing.

Performance





12.07 ms

at 2^{15} threads, 10M values

828.5 M values/s

throughput at 2^{15} threads

+37%

over Version 2.2

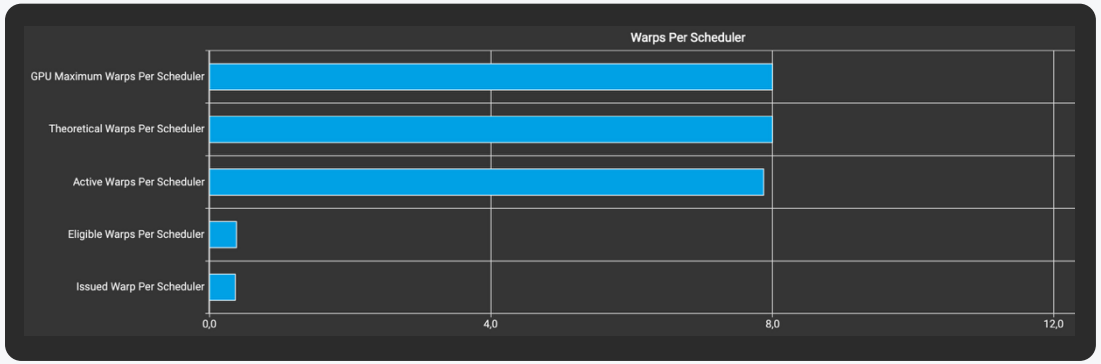
Both bottlenecks removed

Global memory: solved

```
// Found
if (shared_tree[current] == target) {
    found = 1;
    break;
}
// Otherwise move to the correct child
current = (target < shared_tree[current])
    ? (current * 2 + 1)
    : (current * 2 + 2);
```

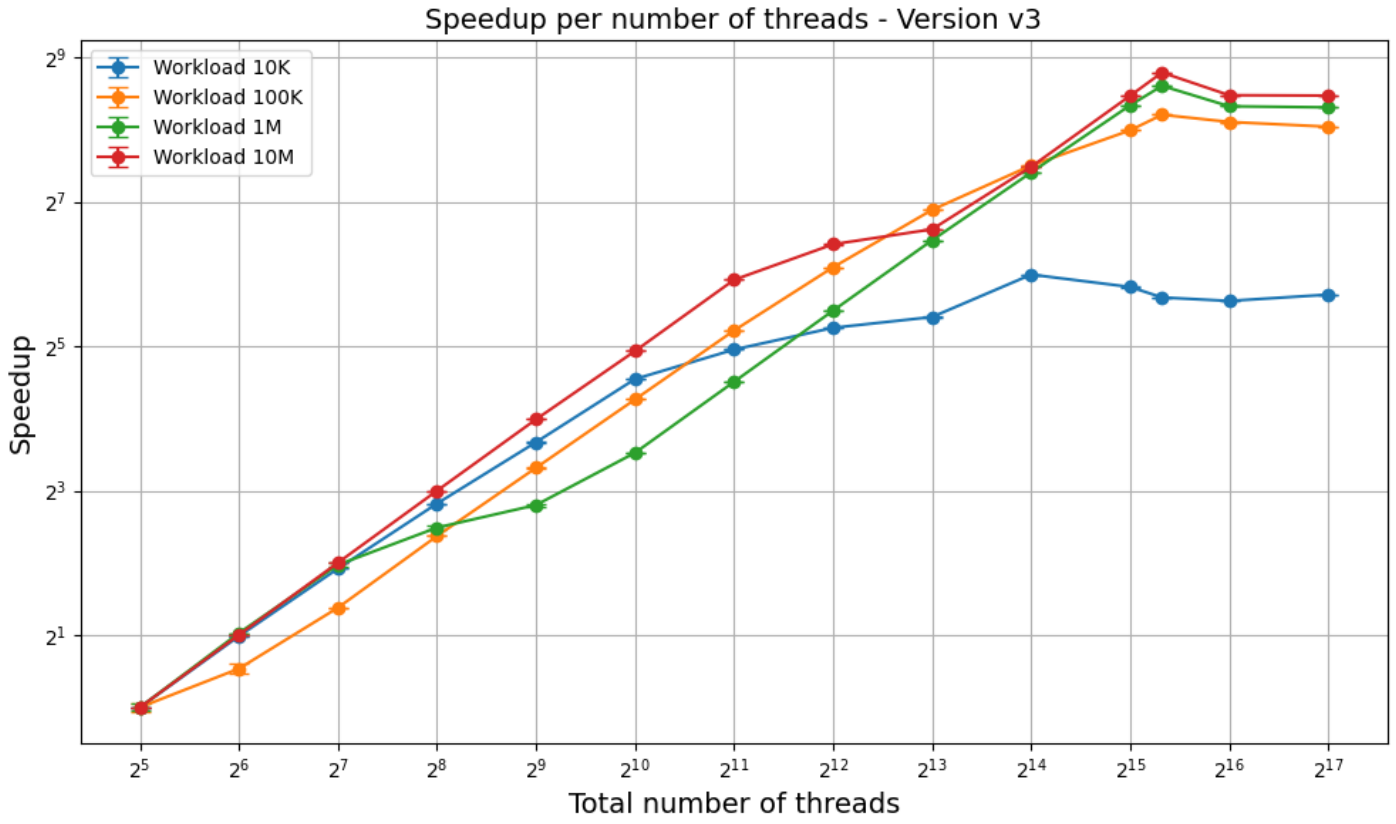
Both reads now hit **Shared** memory; warp stall sampling 1.42% and 1.90%.

Theoretical warps: solved



Theoretical and active warps per scheduler now reach the maximum of 8.

Every SM fully occupied: 40,960 threads



40 SMs × 1,024 threads: the performance increases again at the hardware limit.

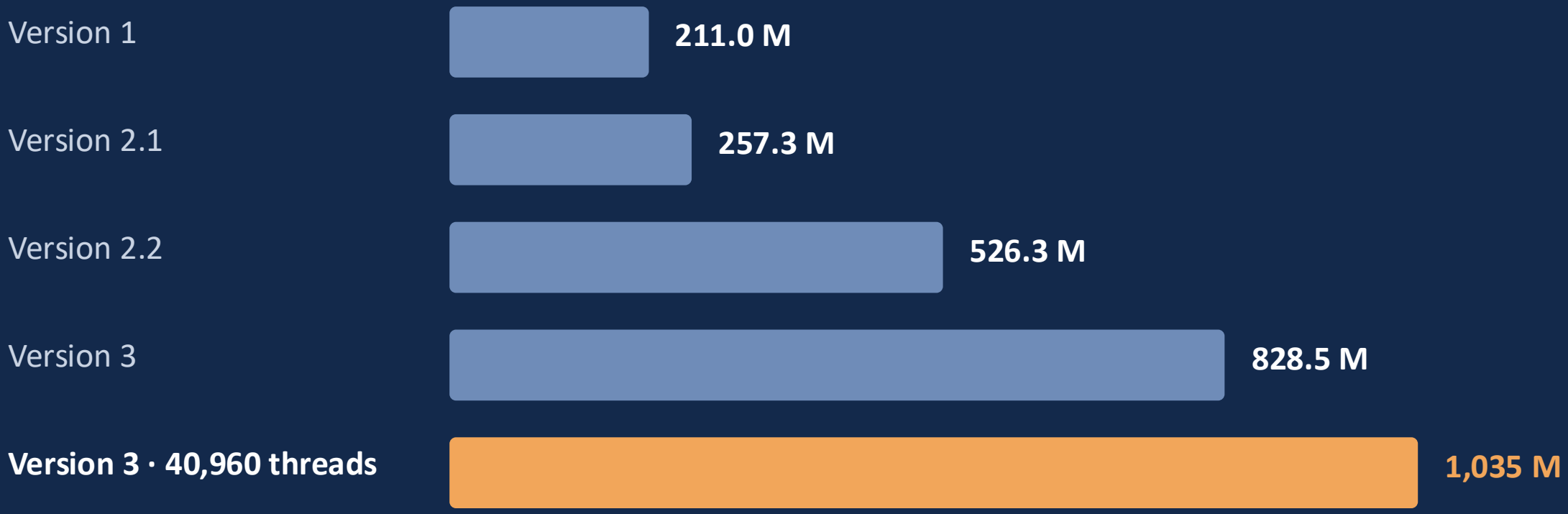
9.66 ms

at 40,960 threads, 10M values

1,035 M values/s

+20% over 2^{15} threads

Throughput: values searched per second



At 2^{15} threads unless noted. Goal: 1 G values/s — **reached**, about 5× the throughput of Version 1.

CONCLUSIONS

Both goals reached

CPU · VERSION 3

14.2 M/s

goal 11 M values/s · 8 threads

GPU · VERSION 3

1,035 M/s

goal 1 G values/s · 40,960 threads

On the CPU

Remove the waiting between threads, then balance the load: equal shares beat clever partitioning.

On the GPU

The memory layout decides: from pointers to a compact implicit array, then into on-chip shared memory.

Method

Every version was driven by the profiler: measure, find the bottleneck, remove it.